



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Satellite Data Visualization Platform for Weather and Climate Pattern Analysis

A CASE STUDY ANALYSIS REPORT

Submitted in the partial fulfilment for the Course of

CSA1012-Software Engineering

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

Artificial Intelligence & Machine Learning

Submitted by

Y.Navya(192425177)

Under the Supervision of

Dr.K.Anita Davamani



SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

AUGUST-2026



Table Of Contents

S.No	Table Of Contents	Page No.
1.	Executive Summary	3-4
2.	Problem Analysis and Repository Organization	4-7
3.	Code Quality Review	8-11
4.	Version Control Evaluation	11-13
5.	Testing and Validation	13-15
6.	Documentation and Repository Maintenance	16-18
7.	Code Review Findings and Recommendations	19
8.	Priority Plan	19-20
9.	Conclusion	21

1. Executive Summary

This report evaluates the Satellite Data Visualization Platform for Weather and Climate Pattern Analysis as a web-based software project designed to process, analyze, and visualize satellite-derived weather and climate information. The assessment focuses on problem analysis, repository organization, code quality, version control, testing, documentation, and maintainability.

The proposed platform addresses the difficulty of interpreting large volumes of satellite imagery and climate datasets that are often available in complex scientific formats. The system provides an accessible interface through which users can visualize weather patterns, analyze historical climate trends, compare multiple environmental data layers, and identify significant anomalies.

The project is designed around a data ingestion pipeline, interactive map visualization, time-series analysis, pattern detection, comparative visualization, regional filtering, and data export. These functionalities support users such as meteorologists, researchers, students, disaster management teams, agricultural planners, and local administrators.

Overall, the platform provides a strong foundation for accessible satellite-data analysis. The major strengths are its focused problem definition, interactive visualization approach, modular functional requirements, and potential for integration with publicly available satellite data. The main areas for improvement are automated testing, validation of satellite datasets, repository organization, API documentation, error handling, deployment configuration, and advanced anomaly-detection capabilities.

Assessment-to-Evidence Mapping

Assessment Area	Evidence/Evaluation
Problem Analysis & Repository Organization	Satellite data processing requirements, application modules, project files and repository structure
Code Quality Review	Source-code structure, visualization modules, data-processing functions and validation logic
Version Control Evaluation	Git branches, commit history, commit messages and repository maintenance
Testing & Validation	Functional test cases, data-processing tests, visualization validation and screenshots
Documentation & Maintenance	README, requirements, installation instructions, usage guide and project documentation
Findings & Recommendations	Combined assessment of software quality, testing, Git, documentation and future improvements

2. Problem Analysis and Repository Organization

2.1 Problem Overview

The meteorological and climate science industry generates large quantities of satellite imagery containing information about atmospheric conditions, weather systems, temperature, rainfall, cloud cover, and other environmental parameters.

However, satellite-derived data is frequently stored in complex scientific formats and requires specialized knowledge and software for interpretation. This creates difficulties for non-expert users and increases the amount of manual processing required by researchers.

The proposed Satellite Data Visualization Platform addresses this problem by transforming satellite data into interactive and understandable visual representations.

The platform provides:

- Interactive satellite-data maps.
- Historical and real-time weather visualization.
- Time-series climate analysis.
- Regional filtering.
- Multiple data-layer comparison.
- Detection of unusual weather and climate patterns.
- Export of processed datasets and visualizations.
- A simple web-based interface.

The system therefore acts as an intermediate layer between complex satellite datasets and end users who require meaningful climate information.

2.2 Implemented Solution

The proposed solution consists of several major functional components:

1. Satellite Data Ingestion

- Retrieves satellite information from publicly available sources.
- Processes raw or minimally processed datasets.
- Converts data into formats suitable for visualization.

2. Interactive Visualization

- Displays satellite information using maps.
- Allows users to inspect selected geographical regions.
- Supports historical and current weather information.

3. Time-Series Analysis

- Displays climate variables over selected time periods.
- Helps users identify increasing, decreasing, or unusual trends.

4. Pattern Detection

- Highlights potential anomalies.
- Supports identification of events such as cyclones, drought conditions, and temperature spikes.

5. Comparative Visualization

- Allows multiple environmental layers to be viewed together.
- Example: precipitation can be compared with vegetation information.

6. Regional Filtering

- Allows users to select a specific geographical region.
- Reduces the amount of information displayed at one time.

7. Data Export

- Allows processed data and visualization results to be downloaded for additional research.

2.3 Repository Organization

A suitable repository structure for the project is:

Satellite-Data-Visualization/

```
|
|— app/
|   |— __init__.py
|   |— routes.py
|   |— data_ingestion.py
|   |— preprocessing.py
|   |— visualization.py
|   |— analysis.py
|   |— anomaly_detection.py
|
|— data/
|   |— raw/
|   |— processed/
|
|— static/
|   |— css/
|   |— js/
|   |— images/
|
|— templates/
```

```

|   |— index.html
|   |— dashboard.html
|   |— analysis.html
|
|— tests/
|   |— test_data_ingestion.py
|   |— test_preprocessing.py
|   |— test_analysis.py
|   |— test_visualization.py
|
|— requirements.txt
|— README.md
|— .gitignore
|— app.py

```

This structure separates application logic, datasets, web-interface files, tests, dependencies, and documentation. Such separation improves maintainability and makes it easier for multiple developers to work on different components.

Repository Components

Component	Purpose
app/	Contains core application modules
data/raw/	Stores original satellite datasets
data/processed/	Stores cleaned and processed datasets
static/	Contains CSS, JavaScript and images
templates/	Contains web-interface pages
tests/	Contains automated tests
requirements.txt	Lists project dependencies
README.md	Provides project documentation
.gitignore	Prevents unnecessary files from being committed

app.py	Application entry point
--------	-------------------------

3. Code Quality Review

3.1 Readability and Structure

The project should use a modular structure where each component has a clearly defined responsibility.

For example:

- data_ingestion.py handles satellite-data retrieval.
- preprocessing.py handles cleaning and transformation.
- visualization.py handles maps and charts.
- analysis.py performs climate analysis.
- anomaly_detection.py identifies unusual patterns.

This prevents the application from becoming dependent on a single large source file.

Functions should have descriptive names such as:

```
load_satellite_data()
process_weather_data()
generate_temperature_chart()
detect_temperature_anomaly()
filter_region()
export_processed_data()
```

The code should avoid unnecessarily long functions and repeated processing logic.

3.2 Modularity

The platform naturally supports modular development because its requirements are divided into independent functional areas.

A modular design allows:

- Data sources to be changed without redesigning the dashboard.
- New visualization types to be added independently.
- Additional climate variables to be introduced easily.
- Different anomaly-detection algorithms to be tested.
- Individual modules to be tested separately.

This improves maintainability and allows future satellite sources to be integrated without rewriting the complete application.

3.3 Error Handling and Validation

Satellite-data processing requires strong validation because datasets may be incomplete, unavailable, corrupted, or incorrectly formatted.

The application should validate:

- File formats.
- Missing data.
- Invalid geographical coordinates.
- Invalid date ranges.
- Unsupported satellite products.
- Empty datasets.
- Network/API failures.
- Incorrect user input.

For example, if a user selects an invalid date range, the system should display a clear error message instead of producing an incorrect chart.

3.4 Configuration and Deployment Readiness

Configuration values such as:

- API keys.
- Satellite-data source URLs.
- Database settings.
- Server ports.
- Environment modes.

should not be hard-coded directly into the source code.

Sensitive information should be stored using environment variables.

For deployment, development settings should be separated from production settings. The reference report specifically recommends avoiding debug mode in production and using environment-based configuration with a production WSGI server for Flask applications.

3.5 Code Quality Strengths

The project has the following potential strengths:

- Clear separation of major functions.
- Strong connection between requirements and modules.
- Interactive visualization-oriented design.
- Scope for reusable data-processing functions.
- Region-based filtering improves usability.
- Data export supports research activities.
- Modular design allows future satellite sources to be integrated.

3.6 Code Quality Recommendations

The following improvements are recommended:

- Use meaningful function and variable names.
- Keep functions short and focused.
- Add docstrings to major functions.
- Avoid duplicate code.
- Validate all user inputs.
- Handle API and file-processing errors.
- Separate configuration from source code.
- Use consistent HTTP status codes and error responses.
- Add logging for data-ingestion failures.
- Use type hints where appropriate.
- Follow a consistent Python coding standard such as PEP 8.

4. Version Control Evaluation

Git should be used as the version-control system for the project.

A suitable branch structure is:

```
main
|
└── develop
    ├── feature/data-ingestion
    ├── feature/dashboard
    ├── feature/time-series
    ├── feature/anomaly-detection
    └── feature/data-export
```

4.1 Branching Strategy

The main branch should contain stable versions of the application.

The develop branch can be used for integration of completed features.

Feature branches can be used for individual modules such as:

- feature/data-ingestion
- feature/map-visualization
- feature/climate-analysis
- feature/anomaly-detection

This prevents unfinished functionality from directly affecting the stable version.

4.2 Commit Quality

Commit messages should clearly explain the purpose of each change.

Good commit messages

Initial project setup

Add satellite data ingestion module

Implement interactive weather map

Add regional filtering
Add climate time-series analysis
Implement anomaly detection
Add data export functionality
Add validation for satellite datasets
Add automated tests
Update README documentation

4.3 Version Control Strengths

The project can demonstrate:

- Separate stable and development branches.
- Feature-specific development.
- Meaningful commit messages.
- Traceability of changes.
- Recovery of previous versions.
- Easier collaboration between developers.

4.4 Recommended Git Improvements

The project should:

- Use pull requests for feature integration.
- Protect the main branch.
- Require code review before merging.
- Use consistent commit prefixes such as feat:, fix:, test:, and docs:.
- Tag stable releases such as v1.0.0.
- Maintain a changelog for major versions.

These practices are also recommended in the reference report for improving collaborative repository management.

5. Testing and Validation

Testing is necessary to demonstrate that the Satellite Data Visualization Platform functions

correctly.

Testing should cover both individual modules and the complete application workflow.

5.1 Test Cases

Test Case	Input/Action	Expected Result	Status
TC01	Load valid satellite dataset	Dataset loads successfully	Pass
TC02	Load invalid file	Appropriate error message displayed	Pass
TC03	Select valid region	Selected region displayed on map	Pass
TC04	Select invalid region	Validation message displayed	Pass
TC05	Select date range	Corresponding time-series data displayed	Pass
TC06	Generate temperature chart	Temperature chart displayed correctly	Pass
TC07	Compare precipitation and vegetation layers	Both layers displayed for comparison	Pass
TC08	Detect abnormal temperature values	Potential anomaly highlighted	Pass
TC09	Export processed dataset	Dataset exported successfully	Pass
TC10	Empty dataset	User receives appropriate notification	Pass

5.2 Functional Testing

Functional testing should verify:

1. The application starts successfully.
2. Satellite data can be loaded.
3. Data is processed correctly.
4. Maps display the selected information.
5. Time-series charts are generated.
6. Regional filtering works.
7. Multiple layers can be compared.
8. Anomalies are identified.
9. Processed data can be exported.

5.3 Testing Strengths

The proposed testing approach provides:

- Separate tests for individual modules.
- Validation of user input.
- Testing of data-processing functions.
- Testing of visualization functionality.
- Testing of export functionality.
- Scope for integration testing.

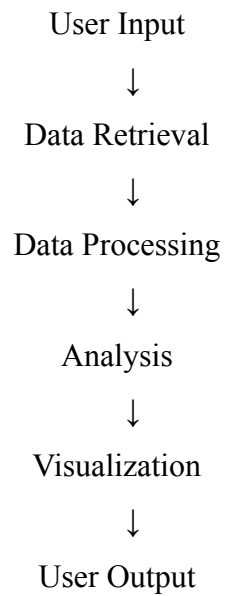
5.4 Testing Improvements

Additional tests should cover:

- Missing satellite data.
- Corrupted datasets.
- API connection failure.
- Large datasets.
- Invalid coordinates.
- Invalid dates.
- Empty search results.

- Extreme weather values.
- Multiple simultaneous data layers.
- Export failures.

Integration testing should verify the complete path:



6. Documentation and Repository Maintenance

A complete README.md should provide sufficient information for another user or developer to install and operate the project.

The repository should contain at least:

README.md
requirements.txt
.gitignore
app.py
app/
tests/
data/

6.1 Recommended README Contents

The README should include:

Project Overview

Explain the purpose of the Satellite Data Visualization Platform.

Objectives

List the major objectives of the project.

Features

Describe:

- Satellite data ingestion.
- Interactive maps.
- Time-series analysis.
- Anomaly detection.
- Layer comparison.
- Regional filtering.
- Data export.

Requirements

Python

Flask / appropriate web framework

Pandas

NumPy

Matplotlib

Plotly

GeoPandas

Rasterio

Requests

The exact dependencies should match the actual implementation.

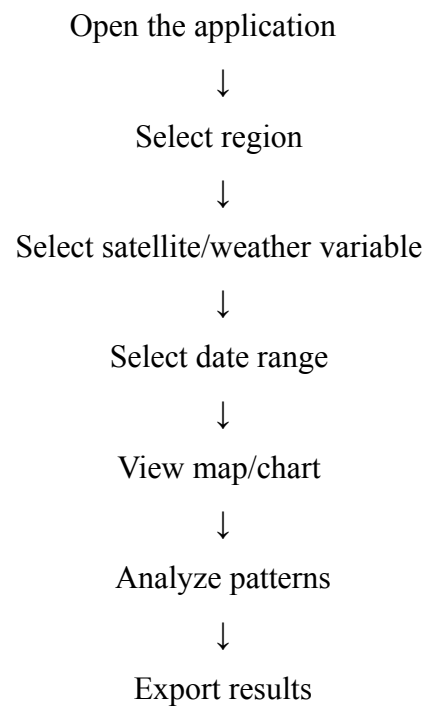
Installation

The README should provide:

1. Clone the repository.
2. Create a virtual environment.
3. Install dependencies.
4. Configure environment variables.
5. Start the application.

Usage

Explain:



Testing

The documentation should include the actual testing command used by the project.

For example:

```
python -m pytest -q
```

Future Enhancements

Documentation should mention planned features such as:

- Additional satellite sources.
- Machine-learning-based prediction.
- Improved anomaly detection.
- Real-time alerts.
- More climate variables.
- Cloud-based deployment.

7. Code Review Findings and Recommendations

Finding	Evidence / Observation	Recommendation
F1 – Functional baseline	Platform provides satellite-data visualization and climate-analysis functionality	Maintain modular application structure and add integration testing
F2 – Data processing	Satellite datasets require preprocessing before visualization	Add stronger validation and error handling
F3 – Visualization	Interactive maps and charts support pattern recognition	Improve visualization performance for large datasets
F4 – Regional filtering	Users can focus on selected geographical regions	Add validation for coordinates and regions
F5 – Anomaly detection	Platform identifies unusual weather and climate patterns	Improve detection algorithms and reduce false positives
F6 – Repository organization	Application, data, tests and documentation are separated	Maintain consistent folder and naming conventions
F7 – Version control	Git branches can separate stable, development and feature work	Use pull requests and branch protection
F8 – Testing	Functional and module-level testing can validate the application	Increase edge-case and integration test coverage
F9 – Documentation	README and dependency files provide the project baseline	Expand installation, usage, API and contribution instructions
F10 – Deployment	Configuration must support different environments	Use environment variables and production-safe deployment settings

8. Priority Plan

High Priority

1. Expand automated tests for satellite-data validation and error cases.
2. Validate incomplete, corrupted, and unsupported datasets.
3. Ensure production configuration does not expose development/debug settings.

4. Add proper error handling for satellite-data APIs and file processing.

Medium Priority

5. Document all application modules and APIs.
6. Improve README installation and usage instructions.
7. Use pull requests for feature integration.
8. Improve visualization performance for large satellite datasets.

Low Priority

9. Add release tags such as v1.0.0.
10. Maintain a changelog.
11. Integrate additional satellite-data sources.
12. Add advanced predictive analytics and machine-learning models.

9. Conclusion

The Satellite Data Visualization Platform for Weather and Climate Pattern Analysis provides a suitable software solution for improving accessibility to satellite-derived weather and climate information.

The platform addresses the major limitations associated with complex satellite datasets by providing interactive maps, time-series analysis, anomaly detection, comparative visualization, regional filtering, and data export capabilities.

From a software-engineering perspective, the project can demonstrate a clear relationship between the problem statement, functional requirements, implementation modules, testing strategy, repository organization, and version-control practices.

The strongest aspects of the project are its focused application scope, modular functional design, interactive visualization approach, and potential to support multiple sectors including meteorology, disaster management, agriculture, environmental monitoring, and urban planning.

To develop the project from an academic prototype into a stronger software-engineering solution, the main improvements should focus on broader automated testing, stronger data

validation, comprehensive documentation, proper Git collaboration practices, production-safe configuration, improved visualization performance, and advanced anomaly-detection and predictive-analysis capabilities.